

LiteCache: A Query Similarity-Driven, GPU-Centric KVCache Subsystem for Efficient LLM Inference

Jiawei Yi

University of Science and Technology of China

Youhui Bai

University of Science and Technology of China

Shengnan Wang

Independent Researcher

Jia He

University of Science and Technology of China

Pnegcheng Wang

Huawei Technologies Co., Ltd

Xia Zhu

Huawei Technologies Co., Ltd

Ping Gong

University of Science and Technology of China

Zewen Jin

University of Science and Technology of China

Jiaqi Ruan

University of Science and Technology of China

Jiaan Zhu

University of Science and Technology of China

Haibo Wang

Huawei Technologies Co., Ltd

Weiguang Wang

Huawei Technologies Co., Ltd

Cheng Li

University of Science and Technology of China

Abstract

During LLM inference, KVCache memory usage grows linearly with sequence length and batch size and often exceeds GPU capacity. Recent proposals offload KV states to host memory and reduce transfers using top- k attention. But their CPU-centric management of the on-GPU cache and CPU-GPU data movement incurs high overhead and fragments the bulk GPU execution that CUDA Graph relies on.

To close this gap, we observe that adjacent queries within the same attention head exhibit strong directional similarity and retrieve highly overlapping top- k KV states. This insight enables a simple head granularity cache algorithm, QSAC, in which each head reuses its previously cached KV states whenever the current query is sufficiently similar to the prior one. QSAC further simplifies cache management primitives and cuts CPU involvement almost entirely. We develop LiteCache, a KVCache subsystem that incorporates QSAC. LiteCache introduces a GPU-centric synchronization controller and speculative sparse prefetching, enabling fully overlapped data movement and computation. These mechanisms produce a stable and predictable execution pattern that remains compatible with the bulk execution mode required by CUDA Graphs.

Evaluation on two widely-used LLMs indicates that LiteCache achieves comparable accuracy to baselines, while sharply minimizing CPU overhead, fully utilizing PCIe bandwidth, thus improving decoding throughput by 10.7-224.2% on both H100 and A40 GPUs and easily supporting sequence

lengths beyond 1M. We open-source LiteCache at [1].

1 Introduction

In recent years, generative large language models (LLMs) [46] have been widely adopted in real-world applications, such as natural language processing [35], code generation [23], and multimodal tasks [53]. Modern LLMs adopt the Transformer architecture and maintain a key-value cache (KVCache) during autoregressive decoding to store past attention states.

However, KVCache memory footprint scales linearly with sequence length and batch size, often exceeding the GPU HBM capacity. For example, a 512K token input to Qwen2.5-14B produces a KVCache of 93.75 GB, more than $3\times$ the model’s parameter size. As models and context windows continue to grow, the tension between limited GPU memory and expanding KVCache demands increasingly constrains the scalability of LLM inference.

To overcome this limitation, recent KVCache subsystems offload KV data to host memory and dynamically fetch only the portions needed for each decoding step [8, 26, 40, 49, 54]. To further reduce KV data transferred over PCI-e, they incorporate top- k attention, which exploits the sparsity of attention to select only the most relevant KV states for each query. Building on this mechanism, advanced subsystems such as RetroInfer [8] and PQCache [54] further maintain a on-GPU cache managed by general purpose replacement algorithms (e.g., LRU [32] and LFU [39]). This two-tier design alleviates GPU memory pressure and reduces data movement compared

to naive offloading. However, these subsystems still fall far short of the performance achieved by a full attention baseline, due to the following two problems.

First, CPU-side cache management has become a major scalability barrier. Each decoding step triggers CPU-driven lookup, data transfer planning, metadata updates, and on-GPU cache replacement. These operations grow costly with longer contexts, larger batches, and larger cache sizes, leading to substantial CPU overhead and GPU stalls. Empirically, we observe that cache-related CPU time can reach 68% of GPU compute time on an H100 [31] (Table 1), causing GPU to idle while waiting for cache metadata and KV blocks to arrive.

Second, CPU–GPU coordination prevents the use of CUDA Graphs [13], a key optimization for modern inference. High-end GPUs such as NVIDIA H100 offer massive compute throughput, yet kernel launch overhead now dominates execution, reaching up to 60% of end-to-end latency in our measurements. CUDA Graphs can greatly reduce this overhead by capturing a stable GPU execution pattern; however, existing KVCache subsystems introduce dynamic, fine-grained CPU–GPU synchronizations and memory transfers that fragment the GPU pipeline and break graph capture. As a result, state-of-the-art KVCache systems cannot benefit from CUDA Graphs, widening the performance gap between full attention and offloading-based approaches. Bridging this gap requires rethinking KVCache subsystem design from the ground up: reducing cache management to a lightweight, predictable, GPU-centric primitive while retaining the flexibility to serve top- k sparse attention efficiently.

To bridge this gap, we present LiteCache, a KVCache subsystem that uses lightweight, predictable, GPU-centric primitives while retaining the flexibility to serve top- k sparse attention efficiently. At the core of LiteCache is the QSAC algorithm, motivated by our key observation that adjacent queries within the same attention head exhibit strong directional similarity and retrieve highly overlapping top- k KV states. This stable intra-head locality allows cache decisions at the granularity of heads rather than blocks. QSAC eliminates list-based metadata, stores only the previous query per head, and performs lookup with a single GPU kernel that checks similarity. When similarity is high, the head reuses its cached KV states with no block-level metadata updates or merge operations, greatly simplifying cache management and reducing CPU involvement.

Building on this lightweight algorithm, LiteCache introduces a set of system-level mechanisms that address the CPU–GPU coordination challenges of existing KVCache subsystems. LiteCache leverages the direction of each query to predict which top- k KV states are likely to be needed and issues speculative sparse prefetching from CPU memory. These transfers proceed concurrently with GPU execution, allowing data movement and computation to overlap. To maintain a GPU-driven pipeline and avoid frequent CPU intervention, LiteCache adds a GPU-side synchronization controller that

polls the completion state of data transfers entirely on the GPU. This removes the fine-grained CPU orchestrations that fragment execution and break CUDA Graph replay. Together, these mechanisms allow LiteCache to execute sparse attention and KVCache offloading with a smooth, static, and bulk GPU execution pattern, enabling high throughput decoding on modern accelerators.

We implemented LiteCache atop FlashInfer [52] and Transformers [48], and open-sourced the system at [1]. We conduct an extensive evaluation using two major LLM families (Llama3-8B [34] and Qwen2.5-14B [37]) across two GPU platforms (NVIDIA A40 and H100), covering a range of sequence lengths from 4K to 1M. Compared to state-of-the-art KVCache subsystems, LiteCache delivers substantial performance gains, improving decoding throughput by 67.0–224.2% on H100 and 10.7–60.5% on A40, while achieving accuracy comparable to full attention on both LongBench [4] and RULER [21]. This shows that combining similarity-aware caching, GPU-centric execution, and model-driven design can overcome long-standing KV-cache bottlenecks and enable scalable long-context inference on modern accelerators.

2 Background and Motivation

2.1 LLM Decoding Inefficiency

Modern large language models (LLMs) are built upon the Transformer architecture, whose attention mechanism enables the model to capture contextual relationships across input tokens [28, 34, 37, 46]. Among the many variants of attention, multi-head attention (MHA) [46] is the most widely adopted. MHA employs h parallel attention heads that jointly process the input. Given input hidden states $\mathbf{X} \in \mathbb{R}^{n \times d}$, where n is the sequence length and d is the hidden dimension, each head i uses its own projection matrices $\mathbf{W}_Q^i, \mathbf{W}_K^i, \mathbf{W}_V^i \in \mathbb{R}^{d \times d_k}$ to obtain the Query, Key, and Value representations $\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i \in \mathbb{R}^{n \times d_k}$ with $d_k = d/h$. The attention for head i is then computed as $\mathbf{O}_i = \text{softmax}(\mathbf{Q}_i \mathbf{K}_i^\top / \sqrt{d_k}) \mathbf{V}_i$. In the end, the outputs of all heads are concatenated and projected through $\mathbf{W}_O \in \mathbb{R}^{d \times d}$ to form the final attention output. Furthermore, recent LLMs adopt grouped query attention (GQA) [28, 34, 37], which reduces memory and computation by allowing multiple query heads to share a single pair of KV projections.

Popular transformer-based models, such as GPT [6] and Llama [45], employ a decoder-only structure. Each inference request is logically divided into two stages: the *prefill* stage and the *decoding* stage. The prefill phase processes the full prompt in parallel to generate the first token, while storing intermediate results of computed keys and values, referred to as the *KVCache*. The decoding stage then uses KVCache to autoregressively generate new tokens, processing only one token at a time due to the constraints of autoregressive generation.

LLM inference faces significant decoding bottlenecks stemming from the KVCache. First, the process is memory constrained [38] because the storage required for the KVCache grows linearly with both batch size and input sequence

length. As a result, its memory footprint can easily exceed the high bandwidth memory (HBM) capacity of even high-end GPUs such as H100. For instance, in Qwen2.5-14B-Instruct-1M [37], a 512K token input produces a KVCache of 93.75 GB, which is $3.3\times$ larger than the model’s 28 GB parameters. Second, during each decoding step, the model must retrieve the full set of previously stored Key and Value vectors (denoted by KV states), leading to substantial data movement overheads that also scale with sequence length and batch size. These bottlenecks are further exacerbated by emerging applications that naturally involve long contexts, such as multi-round dialogue [15], document summarization [24], and code understanding [5]. Meanwhile, modern inference systems increasingly favor large batch sizes to maximize throughput [25], compounding the memory and bandwidth pressure imposed by the KVCache.

2.2 KVCache Subsystems

KVCache subsystems is a critical component of modern LLM inference and have attracted growing research attention [25]. To mitigate GPU memory pressure, various systems offload KVCache from GPU HBM to the much larger host memory, thereby supporting longer contexts and larger batches than GPU capacity alone would allow [26, 40]. In parallel, another line of work aims to reduce the bandwidth cost of loading KVCache during decoding. Top- k attention exploits the intrinsic sparsity of attention [20, 38] by retrieving only the most relevant KV states, achieving near lossless model quality while significantly lowering data transfer volume. Advanced algorithms such as Quest [44], HATA [19], and Loki [41] focus on efficiently identifying the critical KV states. To this end, they generate compact *retrieval metadata* derived from the KCache, such as token block descriptors [44], hash codes [19], and critical channels [41], to facilitate top- k KV selection.

More recent KVCache subsystems integrate both offloading and top- k attention, and additionally use spare GPU memory as a cache for frequently accessed KV data. Representative systems such as RetroInfer [8] and PQCache [54] adopt a unified two tier architecture [8, 26, 40, 49, 54]. As illustrated in Figure 1, the full KVCache resides in CPU-side *KVCache Store*, while GPU maintains a *Cache Buffer* that holds the hottest KV blocks. KV data is managed at block granularity, which typically corresponds to tens to hundreds of tokens.

For managing on-GPU cached data, these systems employ standard cache replacement policies, including LRU [32], ARC [27], and LFU [39]. In detail, each KV block is associated with a list structure used for replacement decisions (insertion, promotion and eviction), and an index structure (hash map) that locates blocks in CPU or GPU memory.

Cache-driven inference workflow. For every query, the GPU first computes the top- k KV indices required for the current attention operation based on the retrieval metadata produced by the top- k attention algorithm. These indices are then passed to the CPU, which performs a block-level *CPU lookup* in the

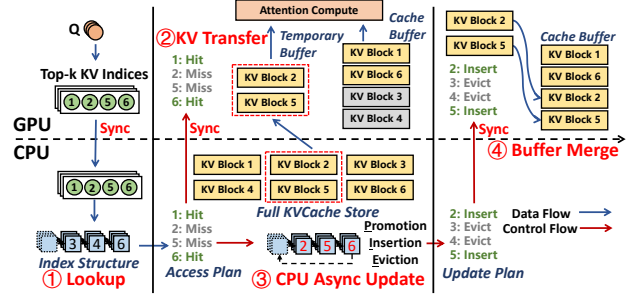


Figure 1: Overview of RetroInfer [8]’s KVCache subsystem design. Note that PQCache [54] follows a similar cache handling workflow but performs cache lookup on GPU.

index structure to determine their locations (①). The lookup generates an access plan that specifies the location of each required block. Blocks found in the GPU cache buffer are marked as hits (e.g., blocks 1 and 6), while those residing in host memory are marked as misses (e.g., blocks 2 and 5). For misses, the CPU gathers the corresponding KV blocks from the full KVCache store and *transfers* them to the GPU over PCIe (②). The GPU places the incoming data into a *temporary buffer*, which is merged with cache-hit data and consumed by the attention computation. Meanwhile, the CPU-side workflow *updates* the list structure using the hit and miss information recorded in the access plan (③). At step-④, the updated list structure then produces a *buffer update plan* that instructs the GPU to merge the KV data in the temporary buffer into the cache buffer (e.g., blocks 2 and 5) and evict out-of-dated data (e.g., blocks 3 and 4).

2.3 Problems and Motivations

The performance of KVCache subsystems plays a vital role in determining the overall efficiency of LLM inference, particularly decoding throughput and latency. To this end, we evaluate the per-step decoding latency of representative KVCache subsystems, including PQCache [54] and RetroInfer [8], compared against a strong full attention baseline (denoted FullAttn) where the entire KVCache is stored in GPU HBM and top- k attention is disabled. We additionally evaluate a variant enhanced with CUDA Graphs (denoted FullAttn+CuGraph), an important optimization that reduces kernel launch overhead on high end GPUs [13]. Note that current KVCache subsystems do not yet integrate CUDA Graphs, and incorporation is nontrivial; we discuss these challenges in Section 2.3.2. All experiments benchmark Llama3-8B [34] on NVIDIA H100 and A40 GPUs across varying sequence lengths.

Surprisingly, Figure 2 shows that both RetroInfer and PQCache underperform the two full attention variants across all evaluated settings, with their sparsity fixed at 10% of total KV states. For example, RetroInfer runs 14–79% and 15–143% slower than FullAttn and FullAttn+CuGraph, respectively. PQCache performs significantly worse, lagging behind by 168–797% and 176–1001% relative to the two base-

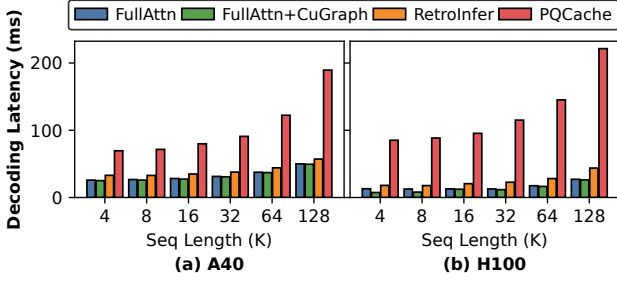


Figure 2: Per-step decoding latency comparison between on-GPU caching systems and full-attention baselines with varied sequence lengths.

Table 1: Ratios of non-overlapped cache-related overhead over GPU kernel time within a single layer across different sequence lengths.

	4k	8k	16k	32k	64k	128k
H100	45%	51%	51%	59%	68%	63%
A40	31%	32%	33%	32%	41%	47%

lines. While enabling CUDA Graphs provides little benefit on the lower end A40 GPU, where kernel launching’s overhead is moderate, the impact becomes substantial on the high end H100 GPU. For instance, FullAttn+CuGraph achieves a $1.01\text{--}1.75\times$ speedup over its non-graph counterpart. Consequently, the performance gap between existing KVCache subsystems and FullAttn+CuGraph widens significantly on modern high performance GPUs. These inefficiencies lead us to conduct a deeper investigation, through which we identify two root causes and motivate a new KVCache subsystem design. Since PQCache consistently underperforms RetroInfer, we focus our subsequent analysis on RetroInfer.

2.3.1 High on-GPU Cache Management Overhead

One key design philosophy behind modern KVCache subsystems is to improve the efficiency of the on-GPU cache so as to increase cache hit ratios and reduce the amount of data transferred over PCIe. However, these benefits come at the cost of substantial CPU-side cache management overhead. As illustrated in Figure 1, each decoding step triggers four major CPU assisted cache operations, namely, `lookup`, `transfer`, `update`, and `merge`, which collectively replenish the on-GPU cache. Among these, `update` operations can overlap with data transfer and GPU attention computation, whereas the remaining operations lie directly on the critical path of every inference request.

To quantify this overhead, we compare the non-overlapped cache management cost against GPU computation time (Table 1). On A40, the ratio of cache-related time to GPU kernel execution ranges from 31% to 47% and increases with sequence length. This problem becomes far more pronounced

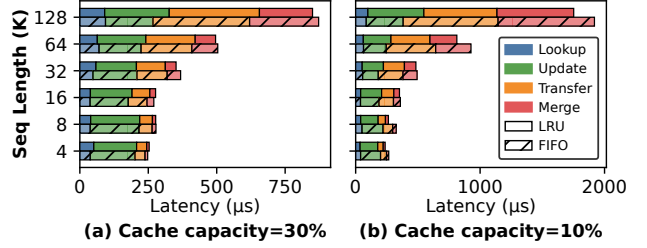


Figure 3: Cache-related timecost breakdown results of FIFO and LRU algorithms with different on-GPU cache sizes. H100 and RetroInfer are used.

on the H100, e.g., at a 128K sequence length, cache related overhead accounts for up to 63% of the corresponding GPU compute time. This reflects the substantial periods during which the GPU remains idle, waiting for cache metadata and KV data to arrive from the CPU.

We further break down this overhead using RetroInfer’s LRU-based policy on H100, with the cache size set to 30% of the full KVCache. Figure 3 (a) shows that `update`’s time-shares are 17-66%, while `lookup` contributes only a small fraction (11-20%), with both increasing modestly as sequence length grows. The high `update` cost stems from RetroInfer’s list based metadata structure. With a block size of 8 tokens, a 30% cache size for 128K sequence requires each attention head to maintain a list of 4.8K entries. Promotions and evictions therefore require traversing long lists, resulting in substantial CPU overhead. In contrast, `transfer` and `merge` costs rise sharply with sequence length. Although LRU maintains a cache hit ratio above 92%, longer contexts inevitably introduce more cache misses, causing `transfer` and `merge` to constitute a large fraction of total overhead at 128K.

A natural question is whether simpler cache algorithms can alleviate this overhead. Unfortunately, replacing LRU with FIFO offers no improvement. Figure 3 (a) depicts that FIFO resembles LRU across all settings and even performs slightly worse beyond 64K tokens. This is due to FIFO’s lower cache hit ratio (about 90%, roughly 2% lower than LRU), which increases miss induced `transfer` and `merge` overhead. Another potential optimization is increasing the block size to reduce metadata. However, enlarging RetroInfer’s block size from 8 to 128 produces no throughput gains. Although the overhead of `lookup` and `update` shrinks by under 10%, the reduced cache hit ratio increases `transfer` and `merge` costs enough to negate any benefit.

We also evaluate a more memory-constrained scenario by reducing the on-GPU cache size from 30% to 10%. Figure 3 (b) shows that cache related overhead nearly doubles. The distribution across the four steps remains similar, but all steps except the negligible `lookup` grow more pronounced. LRU’s hit ratio drops to around 70%, while FIFO falls below 60%, leading to substantially more cache misses and correspondingly higher `transfer` and `merge` costs.

Hint 1: The on-GPU cache management for LLM KVCache subsystems requires **lightweight** designs that jointly consider cache granularity, hit ratio, miss penalty, and the unique access patterns of LLM decoding, rather than relying on generic cache algorithms.

2.3.2 Difficulties of CPU-GPU Cooperation

LLM inference relies on close CPU–GPU cooperation, where the GPU performs nearly all computation, but the CPU remains responsible for launching every kernel and orchestrating data movement. As GPU compute capacity scales rapidly, this division of labor has become increasingly imbalanced. For example, while H100 delivers a $6.6\times$ increase in FLOPS over A40, CPUs typically paired with them improve by only $1.29\times$. This widening gap makes CPU overhead a dominant limiter in modern inference pipelines. The cost of launching kernels is particularly severe. In our H100 experiments, kernel launch overhead alone accounts for up to 60% of end to end decoding latency.

To mitigate this gap, NVIDIA introduced CUDA Graphs [13], which capture a fixed topology of GPU operations including kernels, data movements, and their data dependencies into a directed acyclic graph that can be precompiled into a reusable executable. During inference, this enables the entire forward pass of an LLM to be launched through a single `cudaGraphLaunch` invocation, rather than dozens or hundreds of fine-grained kernel launches. Figure 2 shows that enabling CUDA Graphs yields substantial performance improvement on high-end GPUs such as H100.

However, CUDA Graphs impose strict constraints and require a static operation topology, stable tensor shapes and memory addresses, and the absence of any CPU side synchronization or dynamic control flow. As shown in Figure 1, each decoding step requires the CPU to inspect the cache state, determine which KV blocks must be fetched from host memory, wait for their arrival through asynchronous copies, and then inform the GPU the completion of data movement and which buffer slots to use. This procedure introduces many `cudaMemcpyAsync` calls, copy events, and CPU-side synchronizations into the GPU pipeline, creating fragmentation and preventing the entire iteration from being captured as a stable CUDA Graph.

Hint 2: High-end GPUs favor large, static, and self-contained GPU execution pipelines. An efficient KVCache subsystem must therefore avoid CPU-driven, fine-grained cache orchestration that produces fragmented operations and synchronization points incompatible with CUDA Graphs.

3 Overview

3.1 Key Observations

Rather than relying on temporal locality of accessed top- k KV states as conventional cache algorithms do, our goal is to design a new cache algorithm that simplifies the decision of which top- k KV states are hot and how they should be

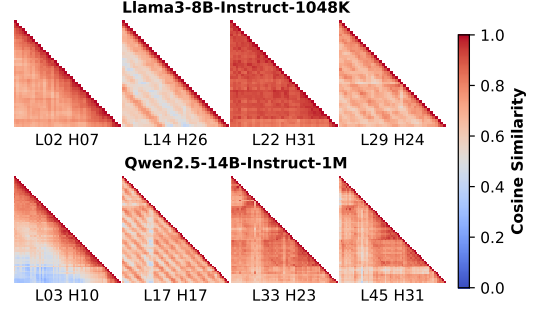


Figure 4: The cosine similarity distribution among query vectors. Evaluated on the decoding phase of a random sampled sequence. "L02 H07" means layer2 and head7. Adjacent queries (near the diagonal) exhibit high cosine similarity.

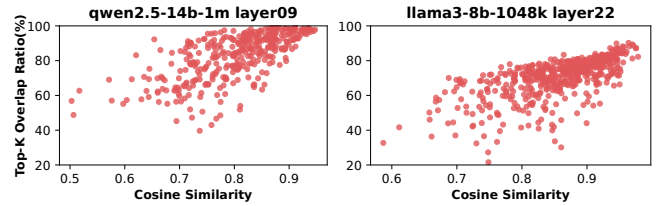


Figure 5: Correlation of the cosine similarity of adjacent query pairs and the overlap ratio of their top- k KV data. Evaluated on sequences randomly sampled from RULER-128K, measured at head granularity. Top- k ratio=10%.

replenished in the on-GPU cache. Our design is motivated by two LLM-specific observations, which together reveal an important property: *adjacent queries from the same attention head tend to retrieve highly similar top- k KV states.*

First, note that the recent proposed advanced LLMs, such as Qwen [37], Llama [45], etc., adopt the rotary position embedding (RoPE) technique [42], under which the queries with adjacent positions will be rotated at similar angles. Consequently, we observe that the adjacent query vectors from the same attention head exhibit high cosine similarity, defined for any two query vectors q_i and q_j as

$$\text{CosSim}(q_i, q_j) = \frac{\text{dot-product}(q_i, q_j)}{\|q_i\| * \|q_j\|},$$

A higher cosine similarity indicates closer alignment in vector direction. Using Qwen2.5 and Llama3, we compute cosine similarities for all query pairs within the same inference request and the same head. As shown in Figure 4, query pairs located near the diagonal, corresponding to queries generated in adjacent decoding steps, consistently show high cosine similarity.

Second, this similarity is strongly reflected in the KV states they retrieve. In Figure 5, we correlate cosine similarity of adjacent queries with the overlap ratio of their retrieved top- k KV states. Both Qwen2.5 and Llama3 exhibit a clear positive correlation, where higher cosine similarity leads to greater overlap in the retrieved KV states. Especially,

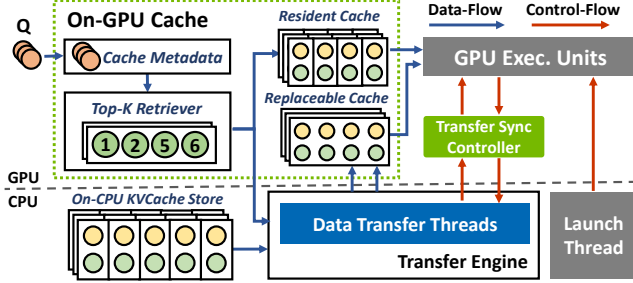


Figure 6: System architecture of LiteCache.

when cosine similarity exceeds 0.9, the two queries share the vast majority of the top- k KV states. We observe the same phenomenon in other widely used models, including DeepSeek [12], GLM [18], and GPT [33], and provide additional evidence in the Appendix.A.

In fact, this phenomenon is in line with the top- k selection mechanism. Note that given a target query q , top- k attention needs to compare the qk scores between the query q and all the keys. For an arbitrary k_i , the qk score is measured by

$$\text{dot-product}(q, k_i) = q^\top k = \|q\| * \|k_i\| \cos(q, k_i). \quad (1)$$

Though the qk score is related to both the vector direction and the vector length of q , the relative ordering of qk scores between q and all keys $\{k_i\}$ is independent of $\|q\|$ since $\|q\|$ is shared. As a result, two queries with similar vector directions correspond to similar ordering of qk scores and the retrieved KV states should overlap to a large extent.

3.2 Design Rationale

The above strong intra-head similarity of adjacent queries motivates us to design a head-wise KVCache mechanism, which is called query similarity-based approximate cache (or short, QSAC). It reuses KVCache entries across decoding steps whenever query directions remain similar, dramatically reducing cache management overhead.

QSAC eliminates the list based metadata structures used in prior work. Instead, each head retains only the query vector from the previous decoding step as its metadata, resulting in a single lightweight entry per head. Cache lookup becomes a cosine similarity check, implemented as a single efficient GPU kernel. Cache update simply replaces the previous step’s stored query with the current decoding-step one upon cache miss, implemented as another GPU kernel. Because each head either keeps its entire top- k KV data in GPU memory or relies entirely on CPU memory, the system no longer requires merge operations. In the end, our new design reduces cache management to two simple GPU-side operations with no CPU involvement, enabling a much more efficient and predictable cache pipeline. Furthermore, this GPU-centric design makes QSAC more friendly with CUDA Graphs’ execution mode. The detailed design of QSAC refers to Section 4.

To realize the benefits of the QSAC, we must address two key technical challenges.

Challenge 1: Balancing accuracy and performance under similarity based reuse. QSAC relies on a similarity threshold to decide when a query can safely reuse the cached head level KV data from the previous step. A low threshold allows aggressive reuse but risks degrading inference accuracy. A high threshold increases accuracy but leads to frequent cache misses, reducing performance. Determining an appropriate threshold that achieves both accuracy and efficiency is therefore nontrivial. Additionally, not all queries exhibit strong similarity even when their retrieved KV data remain important. These corner cases must be handled carefully to prevent accuracy loss without compromising cache efficiency.

Challenge 2: Managing CPU-GPU coordination under increased data movement. Although QSAC eliminates CPU-side metadata management and shifts most cache logic to the GPU, it does not remove all CPU-GPU interactions. Cache misses still require fetching top- k KV data from CPU memory, and the GPU must be correctly notified when these transfers complete. Moreover, by trading cache hit ratio for significantly lower cache management overhead, QSAC may cause more KV data to be transferred from CPU to GPU. These factors collectively make it difficult to maintain a clean and static execution pipeline suitable for CUDA Graphs.

3.3 Overall System Architecture

To address the above challenges, building on our QSAC algorithm, we introduce our new KVCache subsystem LiteCache that enables more efficient LLM inference, illustrated in Figure 6. The system preserves the familiar two-tier architecture, where the CPU-side KVCache store continues to manage KV data at token or block granularity as determined by the specific top- k algorithm. The kernel launch thread and GPU execution units remain the same as the existing systems such as RetroInfer. Beyond this, our system-wise innovations lie in the following added components that enable lightweight on GPU cache management:

On-GPU Cache maintains two cache buffers on GPU. The first is the *Replaceable Cache*, which stores the top- k KV data at head granularity and is sized to hold the top- k portion of the full KVCache. The second is the *Resident Cache*, which keeps KV blocks that exhibit low similarity yet remain important (Section 4.4), using the same granularity as the CPU-side KVCache store. In addition, the module employs a *Top- k Retriever*, allowing LiteCache to perform efficient KV selection with existing top- k attention algorithms.

Data Synchronization Controller. To be compatible with CUDA Graph, we move synchronization logic from CPU to GPU. The controller performs GPU side synchronization and stalls only GPU execution when needed, while a dedicated CPU thread continuously issues kernel launches. This design avoids fine grained CPU coordination and preserves the static, bulk execution pattern required by CUDA Graph.

Algorithm 1 QSAC

Require: The threshold τ_i for each attention head i .

```
1: Initial: at step 0, each head  $i$  caches  $q_c^i = q_0^i, K_c^i = [k_0^i], V_c^i = [v_0^i]$ .
2: for  $1 \leq t \leq T$  do
3:   Given the states  $q_t^i, k_t^i, v_t^i$  of time step  $t$ .
4:   Compute query similarity  $\text{CosSim}(q_t^i, q_c^i)$ , for each head  $i$ .  $\triangleright$  cache lookup
5:   if  $\text{CosSim}(q_t^i, q_c^i) > \tau_i$  then  $\triangleright$  cache hit
6:     Loading KV from the GPU cache.
7:   else  $\triangleright$  cache miss
8:     Compute  $\text{top}-k$  indices by
9:        $\text{index}_i = \text{TopkRetrieval}(q_t^i, K^i, V^i)$ 
10:    Obtain the  $\text{top}-k$  KV data by
11:       $\hat{K}^i = \text{Gather}(K^i, \text{index}_i)$ 
12:       $\hat{V}^i = \text{Gather}(V^i, \text{index}_i)$ 
13:    Transfer KV data by  $K_c^i = \hat{K}^i, V_c^i = \hat{V}^i$ .
14:    Update query cache by  $q_c^i = q_t^i$ .  $\triangleright$  cache update
15:   end if
16:   Compute the attention output by
17:      $\text{output} = \text{Attn}(q_t^i, K_c^i, V_c^i)$ 
18: end for
```

Data Transfer Engine. This component executes the CPU-to-GPU KV data transfer. It employs several dedicated CPU Data Transfer Threads to move required top- k KV data to the Cache Buffer in GPU HBM via zero-copy operation.

4 Cache Algorithm Design

Here, we present QSAC algorithm that exploits strong intra-head query similarity to eliminate costly cache operations.

4.1 Query Similarity-based Approximate Cache

The main idea is to reuse the KVCache for the adjacent steps when the queries exhibit high cosine similarity. To this end, we need to manage the KVCache in a head-wise manner. Algorithm 1 shows the workflow of our design.

Compared with the existing KVCache subsystems, except the KV data cached on GPU, we further cache a query vector q_c^i as metadata for each head i . In QSAC, the lookup becomes a simple cosine similarity computing between the current query q_t^i and the cached query q_c^i (Line 4). If the similarity exceeds the threshold τ_i , it indicates a cache hit (Line 5). In this case, q_t^i is considered a close neighbor of q_c^i , and the top- k KV data can be directly reused and loaded from Replaceable Cache (Line 6). Otherwise, q_t^i is far different from q_c^i , which means cache miss (Line 7). In this situation, we retrieve the top- k KV states for q_t^i and update both the cache buffer for KV (Line 10-13) and query metadata (Line 15). Note that, QSAC does not include the buffer merge step, because the required KV states per head are obtained totally from the GPU if cache hit, and from the CPU if cache miss, rather than using

a CPU-GPU KVCache mixture as in the existing systems.

It is worth noting that we update the query only on cache misses. This ensures alignment between the query and the cached KV data. Updating the query at every step without refreshing the KV data would gradually introduce mismatches between the cached KV and the query representations, ultimately degrading inference accuracy.

4.2 GQA-Aware Similarity Aggregation

To align with the widely-used GQA, data loading and caching must operate at KV-head granularity. However, cosine similarity computation occurs per query head. As a result, an effective intra-GQA similarity aggregation method is required to bridge the gap between KV heads and query heads.

In order to apply our method to GQA scenarios, we need first define the importance of each query head. Though [50] only gave the method to compute the importance score of KV heads, we show that such a method can also be utilized to compute the importance score of query heads, with a slight modification. The detailed process is given in Appendix.B. Now we can give our intra-GQA aggregation method based on the query head importance. To ensure accuracy, we adopt a conservative aggregation manner that follows the two rules:

- The query head with larger importance score should account for a higher proportion during the aggregation.
- The aggregation result should be mainly determined by the query heads with low cosine similarities, especially when they have large importance scores.

Considering a query group including m query heads, denoted by $q_1, q_2, \dots, q_m$, whose importance scores are $s_1, s_2, \dots, s_m$. Assume that at one step, the cosine similarities are $\text{sim}_1, \text{sim}_2, \dots, \text{sim}_m$, respectively, and then the aggregated similarity s is expressed by

$$s = \frac{s_1 + s_2 + \dots + s_m}{\frac{s_1}{\text{sim}_1} + \frac{s_2}{\text{sim}_2} + \dots + \frac{s_m}{\text{sim}_m}}. \quad (2)$$

Such an aggregation satisfies the above two rules, and in addition, when all the query heads have the same cosine similarities, the aggregation result maintains the same value, which is reasonable.

4.3 Similarity Threshold Tuning

The thresholds τ_i are important hyper-parameters that affect both accuracy and performance of the proposed approximate caching method. A lower value of τ_i implies high-frequency reuse of the cached data and low-frequency cache update, contributing to a high inference efficiency. However, in this situation, the query may miss actually important KV states, which leads to accuracy degradation. On the contrary, a higher value of τ_i may achieve a high accuracy but a low efficiency. Therefore, how to configure the reuse thresholds to balance performance and accuracy emerges as the pivotal challenge.

Duo-Attention [50] observed the unequal contributions of KV heads to model accuracy. It also proposed a methodology to profile and quantify this contribution, which produces

head importance scores in the range from 0 to 1, and heads with larger importance scores exhibit higher sensitivity to KV data. Inspired by this, we aim to leverage *head importance* to address the above-mentioned challenges.

Adaptive threshold configuration. We assign a larger threshold to the attention head with larger importance score to protect its effective information. First, we select an upper bound threshold η for all heads. Then, for an attention head i with importance score s_i , its reuse threshold τ_i is computed by

$$\theta^* = \arccos(\eta) \quad (3)$$

$$\theta_i = \lambda_i \theta^* + (1 - \lambda_i) \pi \quad (4)$$

$$\tau_i = \cos(\theta_i) \quad (5)$$

Here, Equation (3) converts similarity threshold upper bound η into an angle threshold upper bound θ . In Equation (4), the coefficient $\lambda_i = s_i^p$, which determines the actual angle threshold θ_i for head i according to its importance. Equation (5) yields the final similarity threshold. In practice, one can set p to 2 or 3 in which case most attention heads have a high reuse rate while the truly important heads can be timely updated, ensuring both effectiveness and efficiency.

4.4 Outlier Handling

The proposed cache policy may encounter some hard-to-reuse heads that involve low hit rates. For such heads, the KVCache in HBM needs to be frequently updated, which will affect the overall efficiency. To tackle this issue, we propose *resident caching*, which retains the full KVCache of the hard-to-reuse heads in GPU HBM. Before detailing this technique, we need to first quantify the *reuse difficulty* of the KV heads:

Quantifying reuse difficulty. We define the reuse difficulty of a KV head, denoted by D_i , as $D_i = \tau_i - (\hat{s}_i - \epsilon)$, where τ_i is the similarity threshold of this KV head and $\hat{s}_i$ is its offline profiled average cosine similarity, which is evaluated on adjacent query pairs generated by tens of random sequences. $\epsilon > 0$ is an error tolerance margin.

Resident caching. We select heads with highest D_i which should be most hard-to-reuse and retain their full KVCache in GPU HBM. This is feasible because the memory size of our cache data buffer equals the top- k KV data size (e.g., only 10% of the total KVCache), leaving substantial free capacity in GPU HBM. In practice, we will set a moderate number of heads with resident caching, according to the specific remaining HBM capacity.

5 CPU-GPU Cooperation

5.1 Cache-Miss Acceleration

In last section, we introduce the cache algorithm QSAC embedded in our system LiteCache, which greatly reduces the cache related overhead. In this section we equip LiteCache with the recently proposed systematic optimization speculative sparse prefetching proposed in InfiniGen [26] to further improve the efficiency.

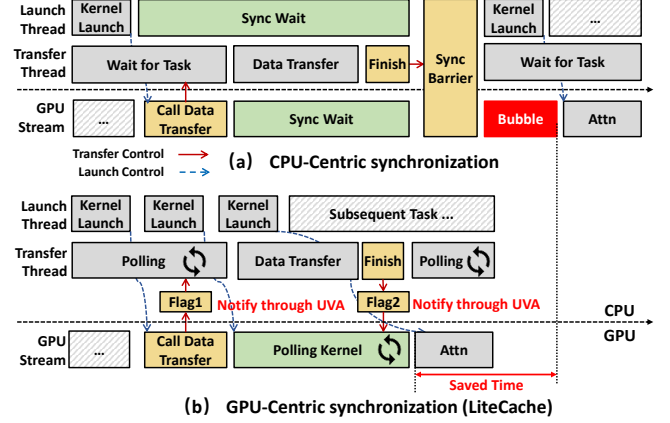


Figure 7: CPU-centric and GPU-centric synchronization.

Speculative sparse prefetching provides a next-layer query predict trough which the next-layer PCIe transfer and current-layer inference computation can be overlapped to some extent. This technology can be well combined with our QSAC cache mechanism. Note that InfiniGen does not involve on gpu cache and it requires to transfer the top- k KVCache of all heads. Such a large amount of transfer is far beyond the inference computation, so that the overlapped transfer time is limited. While in the QSAC cache mechanism, we only prefetch the hard-to-reuse heads to accelerate our cache miss cases. In addition, some hard-to-reuse heads have been addressed by resident caching in Section 4.4, so only a few unaddressed heads requires prefetching. The prefetching data is greatly reduced and the transfer time can be well overlapped.

Speculative sparse prefetching and QSAC utilize the query similarity in adjacent layers and adjacent steps, respectively. The two characteristic are orthogonal and can be well combined. Equipped with the two techniques, LiteCache maximizes the capabilities of GPU chips.

5.2 GPU-Centric Synchronization

To correctly maintain CPU–GPU coordination, existing systems rely on a CPU-centric synchronization strategy. As illustrated in Figure 7 (a), they insert a barrier whenever synchronization is required, which stalls both the CPU and GPU. The CPU is prevented from launching subsequent kernels, creating GPU bubbles. Moreover, because the barrier is executed on the CPU and only becomes visible to the GPU at runtime, it cannot be statically captured by CUDA Graph.

To address this problem, we design a GPU-centric synchronization mechanism. As shown in Figure 7 (b), which blocks the GPU execution instead of the CPU, thus ensures that CPU threads can continuously launch GPU kernels. We decouple kernel launching from data transmission and synchronization tasks by assigning them to a separate CPU thread. The data synchronization is confirmed via flag variables in CPU-GPU shared memory. Specifically, when a transfer is requested, the GPU writes data indices to host memory through Unified Vir-

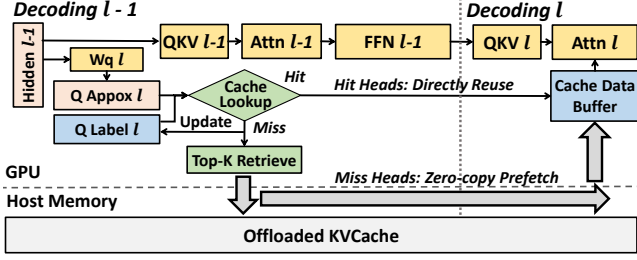


Figure 8: Inference workflow of LiteCache during the decoding phase. For clarity, only processes relevant to cache accessing and host data prefetching for layer l are depicted.

tual Addressing (UVA) [30] and sets the corresponding `Flag1` in shared memory to trigger the transfer engine. Upon completion of the transfer, the transfer engine updates these `Flag2` to notify the GPU. The GPU polls this flag via a `Polling Kernel` and proceeds subsequent kernels only after confirming that the data transfer has finished.

This design prevents CPU-side kernel launches from being stalled by transfer synchronization, and the polling kernel runs entirely on the GPU, allowing it to be statically captured by CUDA Graph. As a result, LiteCache can fully leverage the advantage of CUDA Graph for inference.

6 Put It All Together

We incorporate the above coherent design into a KVCache subsystem LiteCache, to enable efficient LLM inference. This section presents its workflow and implementation.

6.1 LiteCache System Workflow

The workflow of LiteCache can be divided into three phases. **Initialization phase.** During initialization, the KVCache heads are partitioned into HBM-resident and offloaded groups following the strategy described in Section 4. This partitioning is guided by offline-profiled statistics, including head importance, average cosine similarity, and the number of prefetchable heads.

Prefill phase. LiteCache introduces two operations in the prefill phase. First, following the top- k attention algorithm it adopts, LiteCache encodes the KVCache into top- k retrieval metadata as presented in Section 2.2, which incurs minimal overhead compared to other computations in this phase [19]. Second, it offloads partial or full KV heads to host memory according to the initialization partitioning results. This offloading is asynchronously executed alongside GPU computation tasks, with its latency fully hidden by computation.

Decoding phase. During decoding, LiteCache dynamically retrieves and loads the top- k KV data for each step. Figure 8 illustrates this workflow. For layer l , LiteCache begins to retrieve and load its top- k KV data at the beginning of layer $l - 1$. It computes an approximate query vector using the method in InfiniGen [26], and uses it for cache lookup to identify hit heads. For missed heads, LiteCache retrieves new

top- k KV data and launches the prefetch operation. Then, it continues to execute the remaining computation of layer $l - 1$ (attention, FFN). Note that the prefetching mechanism splits the cache update into two phases, namely *query labels update* and *KV data update*. Query labels for missed heads are immediately updated upon the finish of cache lookup, while their KV is gradually updated during prefetching. After the qkv projection of layer l , LiteCache retrieves top- k KV data from Append-only buffer using layer l 's actual query vector. Finally, after prefetching completes, layer l 's attention computation is performed using data from the Cache Data Buffer, which consists of Append-only Cache and Replaceable Cache, as shown in Figure 6.

6.2 Implementation Details

LiteCache is implemented based on FlashInfer [52] and Transformers [48], with 3,417 lines of CUDA/C++ code and 1,555 lines of Python code. Beyond the aforementioned efficient system design and optimizations, LiteCache has also integrated the following additional optimizations to further improve the inference performance:

Zero-copy transfer engine. LiteCache employs a zero-copy engine for CPU-GPU KV transfer that eliminates redundant CPU-side gathering, maximizes PCIe bandwidth, and reduces transfer latency. The engine is built on GDRCopy [29], which exposes GPU memory through the PCIe BAR [2] region and enables direct CPU access via load-store instructions. We use a thread pool to transfer KV data at the granularity of a single K/V vector, whose 128 FP16 elements align with the 64-byte PCIe transfer unit. To further boost zero-copy transfers, the engine exploits AVX SIMD instructions [22] and CPU cache prefetching [3] to improve throughput and mitigate random-access stalls. Overall, this design substantially accelerates KV data movement across the CPU-GPU boundary.

Kernel fusion for cache lookup and query label update. As described in Section 6.1, in LiteCache's workflow, query label update immediately follows the cache lookup. Both operations require to access the incoming query vector and the historical query labels. To avoid redundant HBM accesses and reduce CPU-side kernel-launch overhead, we fuse these two operations into a single GPU kernel.

Hybrid attention kernel. LiteCache's top- k attention computation requires accessing data from both Append-only Cache and Replaceable Cache. To accommodate this hybrid data layout, we implement an attention kernel based on FlashAttention2 [11], which directly reads data from both buffers, without the need for a separate data gathering operation. This design effectively reduces HBM traffic and lowers memory-access overhead.

7 Evaluation

7.1 Experimental Setup

Platform and models. We conduct evaluations on two machines. One machine is equipped with an AMD EPYC 9654

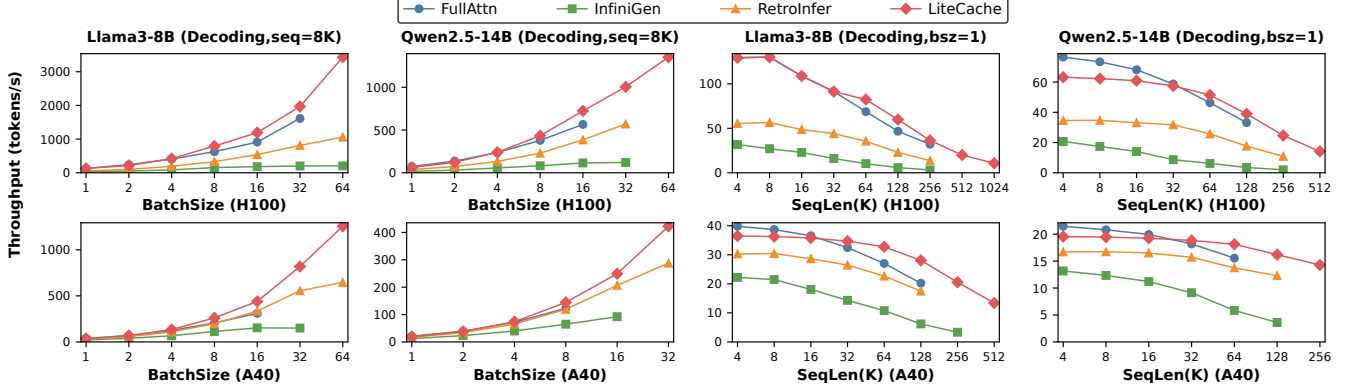


Figure 9: End-to-end decoding throughput across varying batch sizes and sequence lengths.

96-core CPU and an NVIDIA A40 GPU (48 GB) connected via PCIe 4.0 $\times$ 16, while the other is with an Intel(R) Xeon(R) Platinum 8457C 90-core CPU and an NVIDIA H100 GPU (80 GB) connected via PCIe 5.0 $\times$ 16. The main results of two open-source LLMs are shown: Llama3-8B-Instruct-1048K [34] (32 layers, 32 Q heads and 8 KV heads per layer) and Qwen2.5-14B-Instruct-1M [37] (48 layers, 40 Q heads and 8 KV heads per layer).

Benchmarks. We evaluate LiteCache on two widely used benchmarks. The first is *RULER* [21], which consists of 13 subtasks covering retrieval, multi-hop tracking, aggregation, and question answering, with configurable context lengths. The second is *LongBench* [4], a comprehensive benchmark that includes diverse real-world tasks such as summarization, question answering, and code comprehension, spanning a wide range of input text lengths.

Baselines and configurations. We compare LiteCache with SOTA KVCache offloading systems, RetroInfer [8] and InfiniGen [26]. RetroInfer adopts an LRU GPU cache to reduce data transfer, while InfiniGen hides transfer overhead via prefetching. We also include the full-attention implementation without offloading to highlight LiteCache’s advantages. We further consider PQCache [54]. However, it exhibits higher latency in our experiments, mainly due to heavy cache-management overhead and a low cache-hit ratio (Section 2). Given its stronger performance, RetroInfer serves as the primary on-GPU caching baseline in our evaluation. For all offloading baselines, we fix sparsity at 10% of the total KV states, following InfiniGen, a common setting [17, 41, 44]. Unlike PQCache and RetroInfer, LiteCache can work with various top-k attention algorithms [19, 41, 44]. We use HATA [15] (hash bits = 256) for evaluation due to its strong accuracy, but other top-k attention algorithms lead to similar conclusions under the same sparsity, as they only affect the lightweight top-k retrieval step. For LiteCache-specific hyperparameters, we fix $\eta = 0.8$, $p = 3$ in $\lambda_i = s_i^p$, and set ϵ to 0.1 (Llama3-8B) and 0.05 (Qwen2.5-14B). These values are selected from a

small validation sweep and used in all experiments. Details are provided in Appendix.C.

Sink and recent tokens. Suggested by Duo-Attention [50], even for KV heads with zero importance, sink and recent tokens remains necessary for preserving model accuracy. Following this, we persistently keep 4 sink tokens and 64 recent tokens in cache, which is a configuration commonly adopted in such scenarios [8, 51].

CUDA Graph support. Considering the performance benefits of CUDA Graph, we enable it for LiteCache and full-attention. Other baselines, such as RetroInfer and InfiniGen, do not support CUDA Graph by design due to their dynamic and CPU-centric execution patterns, and thus are evaluated without CUDA Graph support.

7.2 Decoding Throughput Comparison

We first present the decoding throughput of different methods on both NVIDIA H100 and A40 GPUs in Figure 9. Some results of baselines are missing due to GPU out-of-memory (OOM) errors of their open-source implementations.

Overall, LiteCache consistently outperforms other offloading-based baselines on both H100 and A40. On H100, it delivers a speedup of 2.08-3.24 $\times$ on Llama3 and 1.67-2.27 $\times$ on Qwen2.5 compared to the leading baseline. On A40, the speedup can be up to 1.61 $\times$ and 1.47 $\times$ on Llama3 and Qwen2.5, respectively. The throughput of LiteCache scales significantly with increasing batch size, consistently outperforming other methods on both models. For short sequences, FullAttn is the top performer because attention is not the primary bottleneck, and other systems incur top-k retrieval overhead. However, as the sequence length grows, LiteCache maintains higher throughput. InfiniGen shows the lowest throughput due to significant data-transfer overheads that cannot be fully hidden, while RetroInfer is increasingly slowed by heavy cache-related overheads.

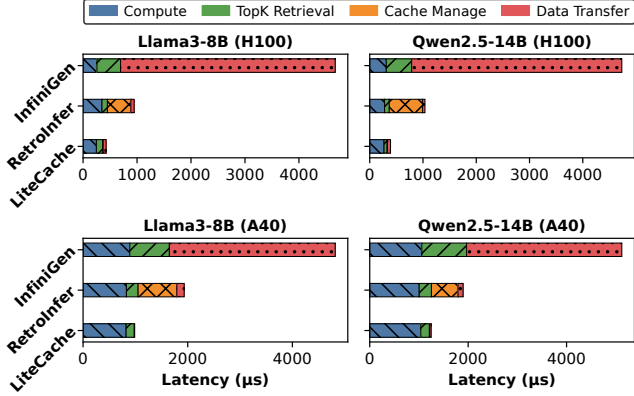


Figure 10: Per-layer decoding latency breakdown, evaluated with batch size=1 and sequence length=128K.

Table 2: GPU cache memory usage (GB) vs. total tokens

Methods	Llama3-8B, #tokens					Qwen2.5-14B, #tokens				
	32K	64K	128K	256K	512K	16K	32K	64K	128K	256K
RetroInfer	1.20	2.40	4.80	9.56	19.12	0.90	1.79	3.59	7.20	14.34
LiteCache	0.53	1.09	2.62	8.19	14.09	0.37	0.74	1.49	3.44	8.03

7.3 Breakdown Analysis

We further breakdown the per-layer decoding latency of LiteCache, InfiniGen, and RetroInfer to better understand the sources of throughput differences. Note that a key efficiency advantage of LiteCache is its compatibility with CUDA Graphs, which substantially reduces kernel launch overhead in the decoding step on high-end GPUs. However, RetroInfer and InfiniGen cannot be integrated with CUDA Graphs due to their dynamic, CPU-centric execution patterns. To keep the breakdown focused on the intrinsic differences in KV-offloading design, we do not include kernel launch overhead in this section. We will present an ablation study of the advantage of CUDA Graph on LiteCache in section 7.6.

As shown in Figure 10, LiteCache achieves a 2.0–4.9 $\times$ speedup on A40 and 2.2–16.1 $\times$ on H100 compared to RetroInfer and InfiniGen, demonstrating its efficiency. Cache management overhead in LiteCache is minimal due to its lightweight caching strategy, whereas RetroInfer spends 28.5–60.2% of decoding time on cache management, and InfiniGen has no cache management by design. The data-transfer overhead is low for both RetroInfer and LiteCache. RetroInfer benefits from a high cache-hit ratio, while LiteCache uses speculative sparse prefetching to overlap the transfer of small miss-induced KV states with computation. InfiniGen also uses speculative prefetching, but the large CPU-to-GPU data volume hinders full overlap.

7.4 On-GPU Cache Statistics

To better understand the effect of on-GPU caching, we compare LiteCache and RetroInfer on the A40 platform. Both

Table 3: Cache hit ratio (%)

Methods	Llama3-8B, SeqLen=8K					Qwen2.5-14B, SeqLen=8K				
	4	8	16	32	64	2	4	8	16	32
RetroInfer	90.4	90.1	90.1	90.0	90.1	92.7	91.3	91.4	91.3	91.4
LiteCache	79.5	74.7	76.7	85.0	80.0	80.6	80.7	80.7	81.5	80.8

Methods	Llama3-8B, BSZ=1					Qwen2.5-14B, BSZ=1				
	32K	64K	128K	256K	512K	16K	32K	64K	128K	256K
RetroInfer	91.2	91.4	91.4	-	-	93.1	93.1	93.4	93.1	-
LiteCache	83.5	82.6	84.0	90.1	88.6	75.8	73.9	65.2	72.5	68.3

Table 4: Inference accuracy results.

Methods	LongBench	RULER			
		16K	32K	64K	128K
Qwen2.5-14B	53.34	94.35	94.48	92.29	88.85
RetroInfer	54.71	94.73	94.41	92.37	89.49
InfiniGen	52.74	92.78	92.19	89.74	85.26
HATA Algo	53.19	94.06	94.13	92.24	88.47
LiteCache + HATA	53.15	94.36	94.22	92.35	88.94
Llama3-8B	41.06	86.07	80.80	76.26	72.96
RetroInfer	41.11	86.36	80.64	76.26	72.73
InfiniGen	40.22	79.70	76.76	72.96	69.37
HATA Algo	41.58	86.56	80.94	76.28	73.87
LiteCache + HATA	41.15	85.65	80.70	75.94	73.44

systems maintain an on-GPU cache to reduce PCIe data transfers. Table 2 shows GPU cache memory usage, while Table 3 reports the cache hit ratios corresponding to the A40-based experiments in Figure 9. LiteCache uses substantially less GPU memory for cache, consuming on average only 47.6% of the memory required by RetroInfer. Although RetroInfer attains a higher cache hit ratio, this causes the heavy LRU-based cache-related overhead, which limits its end-to-end throughput. In contrast, by employing a QSAC caching algorithm together with speculative sparse prefetching, LiteCache achieves an average cache hit ratio of 79.22% while keeping cache-related overhead lightweight, thereby enabling higher throughput with a much smaller on-GPU cache footprint.

7.5 Other Inference Metrics

Accuracy. Table 4 reports the average inference accuracy of different methods on LongBench and RULER. Since LiteCache adopts HATA as its underlying top- k attention algorithm, we additionally include the original HATA top- k algorithm as a baseline to isolate the impact of QSAC and the speculative sparse prefetching strategy on accuracy. Across both benchmarks, LiteCache closely matches the original full-attention models, with a maximum average drop of only 0.42, and it consistently matches or slightly outperforms the HATA algorithm. On RULER, accuracy remains stable as the sequence length increases, indicating robustness on long-context tasks. For both Qwen2.5-14B and Llama3-8B, LiteCache attains accuracy comparable to the best baselines while

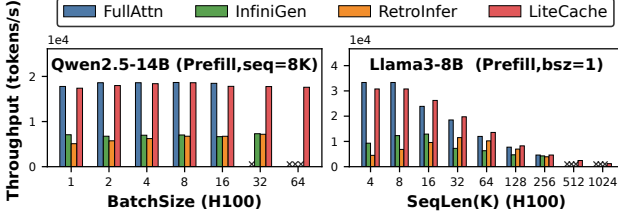


Figure 11: End-to-end prefill throughput across varying batch sizes and sequence lengths on H100.

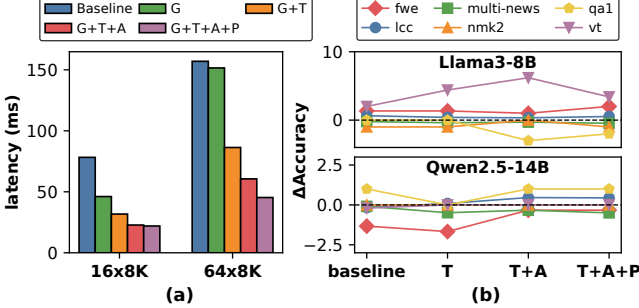


Figure 12: Ablation study of LiteCache’s optimizations. (a) **Latency ablation.** Evaluated on Qwen2.5-15B. (b) **Accuracy ablation.** Evaluated on LongBench (lcc, multi-news) and RULER-128K (nmk2, fwe, qa1, vt).

avoiding the degradation exhibited by InfiniGen. Overall, LiteCache achieves efficient KVCache offloading without compromising inference quality.

Prefill performance. Figure 11 reports the end-to-end prefill throughput on H100. Across both models, with various batch sizes and sequence lengths, LiteCache closely matches FullAttn, whereas InfiniGen and RetroInfer are consistently slower. This conclusion holds for A40 and more batch size / sequence length configurations, which we omit for brevity. In prefill, LiteCache adds lightweight metadata construction and asynchronous KVCache offloading, whose costs are negligible or largely hidden by computation, so its throughput remains close to FullAttn. By contrast, RetroInfer spends substantial time on on-the-fly cache construction, and InfiniGen fails to overlap offloading data transfers with computation in the prefill phase, resulting in lower prefill throughput.

7.6 Ablation Study

Figure 12 presents an ablation study of the optimizations applied in LiteCache, which include: **G** (CUDA Graph), **T** (adaptive Threshold configuration in Section 4.3), **A** (intra-GQA Aggregation in Section 4.2), and **R** (Resident caching in Section 4.4). In addition, **Baseline** is a simple implementation without these optimizations, which only adopts speculative sparse prefetching and QSAC with a fixed reuse threshold for all the KV heads.

First, in Figure 12 (a), we report decoding-step latency on

an H100 GPU with Qwen2.5-14B under batch sizes of 16 and 64 and sequence lengths of 8K, $16 \times 8K$, and $64 \times 8K$ to show the impact of these optimizations at different data scales. Under both configurations, the **Baseline** incurs high latency. Enabling **G** reduces it by 41.1% at $16 \times 8K$ but only 3.5% at $64 \times 8K$, since kernel-launch overhead dominates at small scales but becomes relatively minor at large scales. Second, **T** reduces latency by 31.2% and 43.1% under the two configurations, followed by **A** with reductions of 28.3% and 29.8%. These gains come from both optimizations improving QSAC’s cache hit rate, thereby reducing PCIe transfer overhead. Finally, **R** reduces latency by 3.4% at $16 \times 8K$ and 25.2% at $64 \times 8K$, with a stronger effect at larger scales as transfer and computation latency grow. At $16 \times 8K$, computation can still hide the PCIe transfer cost of cache-missed KV data via prefetching, but as data volume increases, transfer latency starts to dominate and prefetching can no longer fully mask it. In this regime, handling miss-induced, non-prefetchable outliers via **R** becomes crucial for maintaining the throughput stability of LiteCache.

Next, in Figure 12 (b), we demonstrate the impact of all optimizations except **G** on inference accuracy across two models. We use the original HATA algorithm as the baseline and evaluate the change in LiteCache’s inference accuracy after enabling each optimization (denoted as Δ Accuracy, where positive/negative values indicate improvements/degradations). Although LiteCache exhibits accuracy loss on some downstream task benchmarks, most of these are negligible (≤ 1). Furthermore, as described in Section 7.5, the average inference accuracy of LiteCache remains on par with both HATA and FullAttn. Given the significant performance improvements achieved by LiteCache, this trade-off is acceptable.

8 Related Work

KVCache offloading. InflLLM [49], MagicPiG [9] and ShadowKV [43] focus on improving top- k attention algorithms rather than addressing system-level challenges. PQCache [54] and RetroInfer [8] mitigate PCIe transfer volumes through GPU caching, yet they fail to account for the cache management overhead incurred by the CPU. Prefetching approaches, including FlexGen [40] and InfiniGen [26], help alleviate PCIe data transfer latency but face limitations in handling large-scale tasks or entail performance trade-offs. Overall, these methods tend to overlook CPU-side burdens and complexity of implementation. Unlike these, Mooncake [36], LM-Cache [10] and CachedAttention [16] optimize only the prefill stage by leveraging distributed or hybrid memory, and are therefore orthogonal to our work.

Kernel launch optimization techniques have been extensively studied to improve GPU execution efficiency by reducing CPU-side scheduling and launch overhead [11, 13, 14, 55]. Kernel fusion [11, 14] reduces kernel launch overhead by merging multiple fine-grained GPU operations into a single kernel, allowing intermediate data to be reused in registers or

shared memory and reducing redundant global memory accesses. CUDA Graph [13] further mitigates CPU-side scheduling overhead by replaying static GPU execution graphs, amortizing kernel launch costs across iterations. Prior work has shown the effectiveness of CUDA Graph for LLM inference during the decoding phase, where involves many short-lived kernels with low compute intensity [7, 47]. As GPU architectures evolve from A100 to H100 and B200, GPU compute and memory throughput scale far faster than CPU-side scheduling. This widening gap further motivates the use of CUDA Graph execution to mitigate CPU-induced bottlenecks.

9 Conclusion

LiteCache is an efficient KVCache subsystem for improving LLM inference under GPU HBM constraints. Leveraging query similarities, it eliminates cache-related bottlenecks and enables a fully overlapped computation–transfer pipeline. LiteCache improves inference throughput by 67.0–224.2% on H100 and 10.7–60.5% on A40 compared to strong baselines, while preserving inference accuracy with popular LLMs.

References

- [1] Source code of litecache. <https://anonymous.4open.science/r/LiteCache-888D>, 2025. accessed, Dec. 2025.
- [2] AMD. Pcie base address registers. <https://docs.amd.com/r/en-US/pg055-axi-bridge-pcie/PCIe-Base-Address-Registers>, 2023. accessed, Dec. 2025.
- [3] ArmDeveloper. Cache prefetching. <https://learn.arm.com/learning-paths/cross-platform/memory-latency/latency-and-cache-prefetching/>, 2025. accessed, Dec. 2025.
- [4] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A bilingual, multitask benchmark for long context understanding. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3119–3137, Bangkok, Thailand, August 2024. Association for Computational Linguistics.
- [5] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Vageesh D C, Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, Balasubramanyan Ashok, and Shashank Shet. Codeplan: Repository-level coding using llms and planning. *Proceedings of the ACM on Software Engineering*, 1(FSE):675–698, 2024.
- [6] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020.
- [7] Rodrigo Garcia Casasola. Deepseek-v3 + sglang: Inference optimization. <https://verda.com/blog/deepseek-v3-sglang-inference-optimization>, 2025. accessed, Dec. 2025.
- [8] Yaoqi Chen, Jinkai Zhang, Baotong Lu, Qianxi Zhang, Chengruidong Zhang, Jingjia Luo, Di Liu, Huiqiang Jiang, Qi Chen, Jing Liu, et al. Retroinfer: A vector-storage approach for scalable long-context llm inference. *arXiv preprint arXiv:2505.02922*, 2025.
- [9] Zhuoming Chen, Ranajoy Sadhukhan, Zihao Ye, Yang Zhou, Jianyu Zhang, Niklas Nolte, Yuandong Tian, Matthijs Douze, Leon Bottou, Zhihao Jia, and Beidi Chen. MagicPIG: LSH sampling for efficient LLM generation. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [10] Yihua Cheng, Yuhan Liu, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Kuntai Du, and Junchen Jiang. Lmcache: An efficient kv cache layer for enterprise-scale llm inference. *arXiv preprint arXiv:2510.09665*, 2025.
- [11] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations*, 2024.
- [12] DeepSeek-AI, Aixin Liu, Bei Feng, Bin Wang, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model, 2024.
- [13] CUDA Developer. Cuda graph. <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/cuda-graphs.html>, 2025. accessed, Dec. 2025.
- [14] Jiří Filipovič, Matúš Madzin, Jan Fousek, and Luděk Matyska. Optimizing cuda code by kernel fusion: application on blas. *The Journal of Supercomputing*, 71(10):3934–3957, 2015.
- [15] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. Cost-efficient large language model

- serving for multi-turn conversations with cached attention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 111–126, 2024.
- [16] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. Cost-efficient large language model serving for multi-turn conversations with cached attention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 111–126, 2024.
- [17] Yizhao Gao, Zhichen Zeng, Dayou Du, Shijie Cao, Peiyuan Zhou, Jiaxing Qi, Junjie Lai, Hayden Kwok-Hay So, Ting Cao, Fan Yang, and Mao Yang. Seerattention: Learning intrinsic sparse attention in your llms, 2025.
- [18] Team GLM, Aohan Zeng, Bin Xu, Bowen Wang, Chenhui Zhang, et al. Chatglm: A family of large language models from glm-130b to glm-4 all tools, 2024.
- [19] Ping Gong, Jiawei Yi, Shengnan Wang, Juncheng Zhang, Zewen Jin, Ouxiang Zhou, Ruibo Liu, Guanbin Xu, Youhui Bai, Bowen Ye, Kun Yuan, Tong Yang, Gong Zhang, Renhai Chen, Feng Wu, and Cheng Li. HATA: Trainable and hardware-efficient hash-aware top-k attention for scalable large model inference. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Findings of the Association for Computational Linguistics: ACL 2025*, pages 24856–24871, Vienna, Austria, July 2025. Association for Computational Linguistics.
- [20] Ankit Gupta, Guy Dar, Shaya Goodman, David Ciprut, and Jonathan Berant. Memory-efficient transformers via top-k attention. In Nafise Sadat Moosavi, Iryna Gurevych, Angela Fan, Thomas Wolf, Yufang Hou, Ana Marasović, and Sujith Ravi, editors, *Proceedings of the Second Workshop on Simple and Efficient Natural Language Processing*, pages 39–52, Virtual, November 2021. Association for Computational Linguistics.
- [21] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, and Boris Ginsburg. RULER: What’s the real context size of your long-context language models? In *First Conference on Language Modeling*, 2024.
- [22] Intel. Avx-512 instructions. <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-avx-512-instructions.html>, 2017. accessed, Dec. 2025.
- [23] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. A survey on large language models for code generation. *arXiv preprint arXiv:2406.00515*, 2024.
- [24] Huan Yee Koh, Jiaxin Ju, Ming Liu, and Shirui Pan. An empirical survey on long document summarization: Datasets, models, and metrics. *ACM computing surveys*, 55(8):1–35, 2022.
- [25] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with paged attention, 2023.
- [26] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. Infinigen: Efficient generative inference of large language models with dynamic kv cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 155–172, 2024.
- [27] Nimrod Megiddo and Dharmendra S Modha. Arc: A self-tuning, low overhead replacement cache. In *2nd USENIX Conference on File and Storage Technologies (FAST 03)*, 2003.
- [28] MetaAI. Introducing llama 3.1: Our most capable models to date. <https://ai.meta.com/blog/meta-llama-3-1/>, 2024. accessed, Nov. 2025.
- [29] NVIDIA. Magnum io gdrCOPY: Enable faster memory transfers between cpu and gpu with gdrCOPY. <https://developer.nvidia.com/gdrCOPY>, 2021. accessed, Dec. 2025.
- [30] NVIDIA. Cuda c++ programming guide. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/#unified-virtual-address-space>, 2025. accessed, Dec. 2025.
- [31] NVIDIA. Nvidia h100 gpu whitepaper. <https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c>, 2025. accessed, Dec. 2025.
- [32] Elizabeth J O’neil, Patrick E O’neil, and Gerhard Weikum. The lru-k page replacement algorithm for database disk buffering. *Acm Sigmod Record*, 22(2):297–306, 1993.
- [33] OpenAI, Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, et al. gpt-oss-120b and gpt-oss-20b model card, 2025.
- [34] Leonid Pekelis, Michael Feil, Forrest Moret, Mark Huang, and Tiffany Peng. Llama 3 gradient: A series of long context models, 2024.

- [35] Libo Qin, Qiguang Chen, Xiachong Feng, Yang Wu, Yongheng Zhang, Yinghui Li, Min Li, Wanxiang Che, and Philip S Yu. Large language models meet nlp: A survey. *arXiv preprint arXiv:2405.12819*, 2024.
- [36] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation—a kv-cache-centric architecture for serving llm chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 155–170, 2025.
- [37] QwenTeam. Qwen2.5-1m: Deploy your own qwen with context length up to 1m tokens. <https://qwenlm.github.io/blog/qwen2.5-1m/>, 2025. accessed, Nov. 2025.
- [38] Luka Ribar, Ivan Chelombiev, Luke Hudliss-Galley, Charlie Blake, Carlo Luschi, and Douglas Orr. Sparq attention: bandwidth-efficient llm inference. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- [39] John T Robinson and Murthy V Devarakonda. Data cache management using frequency-based replacement. In *Proceedings of the 1990 ACM SIGMETRICS conference on Measurement and modeling of computer systems*, pages 134–142, 1990.
- [40] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*, pages 31094–31116. PMLR, 2023.
- [41] Prajwal Singhan, Siddharth Singh, Shwai He, Soheil Feizi, and Abhinav Bhatele. Loki: Low-rank keys for efficient sparse attention. *Advances in Neural Information Processing Systems*, 37:16692–16723, 2024.
- [42] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- [43] Hanshi Sun, Li-Wen Chang, Wenlei Bao, Size Zheng, Ningxin Zheng, Xin Liu, Harry Dong, Yuejie Chi, and Beidi Chen. Shadowkv: Kv cache in shadows for high-throughput long-context llm inference. *arXiv preprint arXiv:2410.21465*, 2024.
- [44] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: query-aware sparsity for efficient long-context llm inference. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- [45] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- [46] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [47] vLLM. Cuda graphs. https://docs.vllm.ai/en/stable/design/cuda_graphs/, 2025. accessed, Dec. 2025.
- [48] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online, October 2020. Association for Computational Linguistics.
- [49] Chaojun Xiao, Pengle Zhang, Xu Han, Guangxuan Xiao, Yankai Lin, Zhengyan Zhang, Zhiyuan Liu, and Maosong Sun. Inllm: Training-free long-context extrapolation for llms with an efficient context memory. *Advances in Neural Information Processing Systems*, 37:119638–119661, 2024.
- [50] Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. Duoattention: Efficient long-context LLM inference with retrieval and streaming heads. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [51] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*, 2024.
- [52] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. Flashinfer: Efficient and customizable attention engine for llm inference serving. *arXiv preprint arXiv:2501.01005*, 2025.
- [53] Shukang Yin, Chaoyou Fu, Sirui Zhao, Ke Li, Xing Sun, Tong Xu, and Enhong Chen. A survey on multi-modal large language models. *National Science Review*, 11(12):nwae403, 2024.

- [54] Hailin Zhang, Xiaodong Ji, Yilin Chen, Fangcheng Fu, Xupeng Miao, Xiaonan Nie, Weipeng Chen, and Bin Cui. Pqcache: Product quantization-based kvcache for long context llm inference. *Proceedings of the ACM on Management of Data*, 3(3):1–30, 2025.
- [55] Size Zheng, Siyuan Chen, Peidi Song, Renze Chen, Xiuhong Li, Shengen Yan, Dahua Lin, Jingwen Leng, and Yun Liang. Chimera: An analytical optimizing framework for effective compute-intensive operators fusion. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 1113–1126. IEEE, 2023.